

Projeto incremental — Plataforma de Reservas

Luccas Rafael
Programação Orientada a Objetos — Fagammon

Segundo semestre de 2026

Projeto semestral em Java, desenvolvido preferencialmente em dupla, avaliado em marcos executáveis e demonstração. Valor total: 20 pontos.

1 Desafio

Construir uma aplicação Java que administre reservas de recursos durante um período. A aplicação deverá cadastrar participantes e recursos, impedir conflitos, aplicar regras variáveis, acompanhar o ciclo de vida das reservas, oferecer consultas e preservar os dados entre execuções.

A interface poderá ser uma aplicação de linha de comando. Interface gráfica, aplicação web e banco de dados não fazem parte da nota. O foco é a qualidade do modelo, do comportamento, dos testes e da evolução do código.

2 Equipes e autoria

- modalidade preferencial: dupla;
- trabalho individual: permitido;
- todos os integrantes devem participar do histórico e compreender a solução inteira;
- a demonstração poderá incluir pergunta ou pequena alteração individual;
- quantidade de commits ou linhas não será usada isoladamente para medir contribuição.

Divisão rígida como “um fez o código e outro fez o relatório” não demonstra colaboração suficiente. Histórico, decisões, testes e capacidade de explicar o modelo serão analisados em conjunto.

3 Variantes

Cada equipe escolherá uma variante e acrescentará ao menos uma regra própria do domínio. A escolha será confirmada pelo professor para manter complexidade comparável.

Variante	Exemplos de recursos	Regra específica esperada
Salas e laboratórios	sala, auditório, laboratório	capacidade e recursos necessários
Quadras esportivas	quadra, campo, equipamento	modalidade, manutenção e duração
Equipamentos	câmera, notebook, projetor	quantidade, retirada e devolução
Serviços e atendimentos	profissional, serviço, horário	duração, especialidade e agenda
Espaços para eventos	salão, estúdio, área externa	capacidade, adicionais e antecedência

Outra variante poderá ser proposta se possuir recurso disputado, períodos, conflitos e uma regra variável relevante.

4 Requisitos funcionais comuns

RF1 Participantes: cadastrar e consultar o responsável pela reserva, usando identificador estável e dados validados.

RF2 Recursos: cadastrar, ativar, desativar e consultar recursos, incluindo os dados necessários para avaliar uma solicitação.

RF3 Períodos: representar início e fim por objeto de valor capaz de validar e detectar sobreposição.

RF4 Reserva: associar responsável, recurso e período; recusar recurso inexistente, inativo, incompatível ou ocupado.

RF5 Ciclo de vida: representar estados e impedir transições inválidas.

RF6 Cancelamento: aplicar ao menos duas políticas substituíveis, sem condicionais espalhadas.

RF7 Preço ou custo: oferecer ao menos duas formas de cálculo e não usar `double` para valores monetários.

RF8 Consultas: filtrar e ordenar reservas por critérios úteis, incluindo uma consulta com `lambdas` ou `streams`.

RF9 Persistência: salvar e restaurar em formato aberto e inspecionável, como JSON.

RF10 Relatório: produzir uma síntese útil para a variante.

5 Invariantes mínimas

O modelo deverá garantir:

1. início do período anterior ao fim;
2. identificadores e dados obrigatórios não nulos nem vazios;

3. recurso inativo sem novas reservas;
4. ausência de sobreposição ativa para o mesmo recurso;
5. preço, duração e capacidade dentro de limites válidos;
6. transições de estado válidas;
7. reserva cancelada ou concluída sem retorno silencioso ao início;
8. dados restaurados sujeitos às mesmas validações dos dados novos;
9. ao menos uma invariante própria da variante.

6 Evidências orientadas a objetos

O projeto deverá demonstrar, com justificativa:

- objeto de valor imutável como `record` ou classe imutável;
- distinção entre identidade de entidade e igualdade de valor;
- composição entre objetos do domínio;
- contrato polimórfico com duas implementações reais;
- escolha consciente entre interface, classe abstrata, classe concreta, `enum` e tipo selado;
- abstração genérica própria, como `Repositorio<T, ID>`;
- coleções escolhidas pelo contrato do domínio;
- exceções que comuniquem falhas tratáveis;
- ao menos um padrão entre `Strategy`, `Factory` e `Adapter`, aplicado a um problema concreto.

Criar tipos ou padrões apenas para cumprir quantidade não demonstra boa orientação a objetos.

7 Testes obrigatórios

A suíte deverá provar comportamentos relevantes, incluindo:

1. criação de reserva válida;
2. rejeição de período inválido;
3. rejeição de sobreposição e aceitação de períodos compatíveis;
4. transição válida e inválida;
5. comportamento das duas políticas;
6. igualdade e hashing de um objeto de valor;
7. filtragem e ordenação;
8. persistência e restauração;
9. regra específica da variante.

Não haverá meta de quantidade ou cobertura usada isoladamente. Os testes devem executar com `./mvnw test` ou `mvnw.cmd test`.

8 Restrições técnicas

- Java 25 LTS, sem recursos *preview*;
- Maven Wrapper incluído;
- JUnit 6;
- API moderna de data e hora;
- `BigDecimal` ou objeto de valor equivalente para dinheiro;
- UTF-8 e formato aberto para persistência;
- build e testes reproduzíveis fora da IDE.

Dependências pequenas poderão ser usadas quando declaradas no `pom.xml`. Lombok, Spring, Hibernate e frameworks que escondam o mecanismo estudado não serão usados no núcleo avaliado.

9 Documentação esperada

- `README.md`: problema, variante, integrantes, requisitos e comandos;
- `docs/decisoes.md`: decisões e alternativas descartadas;
- diagrama de classes atualizado nos checkpoints;
- `IA_USAGE.md`: usos materiais de IA e verificações realizadas;
- dados de demonstração sem informações pessoais reais.

10 Marcos e pontuação

Data	Marco	Evidência principal	Pontos
25/08	lançamento	enunciado, equipe e repositório inicial	—
06/10	checkpoint 1	modelo, invariantes, testes e build	4
27/10	checkpoint 2	núcleo, polimorfismo, genéricos e consultas	5
10/11	checkpoint 3	persistência, padrão, refatoração e testes	5
24/11	entrega e demonstração	integração, documentação, histórico e domínio individual	6
Total			20

Checkpoint 1 — 4 pontos

Modelagem, responsabilidades e relações	1,5
Invariantes preservadas pelo código	1,5
Testes iniciais relevantes	0,5
Maven, Git e instruções mínimas	0,5

Checkpoint 2 — 5 pontos

Evolução do modelo e das invariantes	1,0
Composição, polimorfismo e genéricos	2,0
Requisitos e tratamento de falhas	1,0
Testes da evolução	0,5
Processo e documentação	0,5

Checkpoint 3 — 5 pontos

Abstrações e padrão justificado	2,0
Comportamento integrado	0,5
Testes de limites e regressão	1,0
Refatoração, legibilidade e baixo acoplamento	1,5

Entrega e demonstração — 6 pontos

Requisitos finais e robustez	1,5
Suíte final e ausência de regressões conhecidas	1,0
Qualidade após integração	0,5
Build, histórico e documentação	1,0
Demonstração, explicação e autoria individual	2,0

11 Uso de inteligência artificial

Ferramentas de IA podem apoiar explicações, alternativas, testes e revisão. A equipe continua responsável por compreender, verificar e defender o que incorporar. Registre os usos materiais em `IA_USAGE.md`, incluindo:

- objetivo do uso;
- parte do projeto afetada;
- como o resultado foi verificado;
- correções ou decisões feitas pela equipe.

Código que a equipe não consegue explicar ou testar não comprova autoria.

12 Início do trabalho

Até a confirmação da variante em 01/09:

1. formar a equipe ou registrar a opção individual;
2. criar o repositório usando o modelo fornecido;
3. executar `./mvnw test`;
4. ler os requisitos e registrar dúvidas;
5. escolher uma variante preliminar e uma regra própria possível.

Não é necessário fechar a arquitetura agora: ela evoluirá com o domínio e as aulas do semestre.